Baladithya Balamurugan
1699864, bbalamur@ucsc.edu

CSE 102 — Fall 2020 – Homework 4

This assignment comprises of five questions and is worth fifty points. It is due on November 22 (10 a.m.) on Gradescope.

Before you begin the assignment, please read the following carefully.

- Read the *Homework Guidelines*.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.
- A subset of the questions may be graded due to limited resources.

I have read and agree to the collaboration policy. – Baladithya Balamurugan, bbalamur@ucsc.edu
Collaborators:

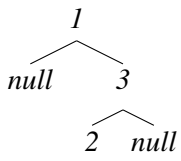
---

1. (10 pts) Let B(n) be the number of different binary search trees containing the n keys 1, 2, . . . , n. For this problem you will develop a dynamic programming algorithm that, given n, calculates B(n). Note that B(1) = 1 since there is only one one-node binary tree. For two nodes B(2) = 2 (either node can be root, and there is only one binary search tree consistent with each choice).

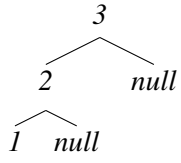
   (a) (1 pt) Calculate by hand B(3).

   Solution.
   $B(3) = 5$.
   Unique BST's for $B(3)$*:*

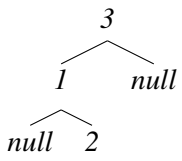
   i.

   ii.

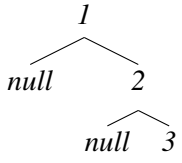
   iii.

   iv.

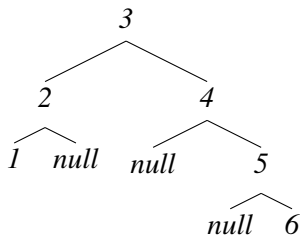
   v.

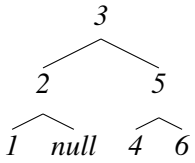
(b) (1 pt) How many n = 6 key binary trees are there with keys $\{1, 2, 3, 4, 5, 6\}$ and key 3 at root (so the left subtree has two nodes, and the right one has 3 nodes) are there?

Solution.
Unique BST's for $B(6)$ (with only root at 3) count 10:

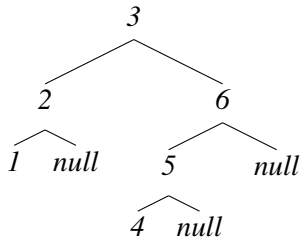
i.



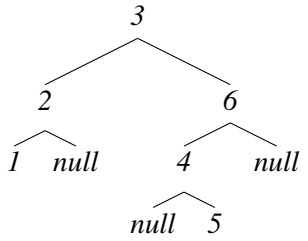
ii.



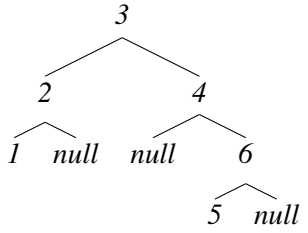
iii.



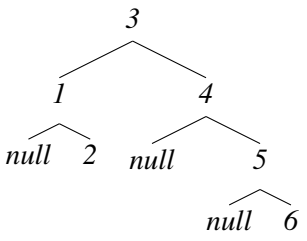
iv.



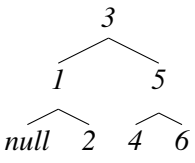
v.



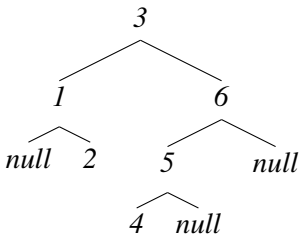
vi.



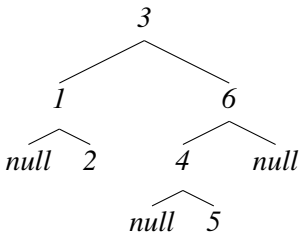
vii.



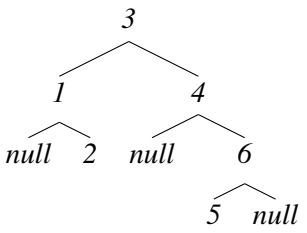
viii.



ix.



x.



(c) (3 pts) Construct a recurrence for B(n), I expect something with a sum. Include boundary conditions in your recurrence; what is a convenient boundary value for B(0)? Briefly justify (formal proof not required) that your recurrence computes the correct value.

Solution.

$$B(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ B(n) + (B(n-j) * B(j-1)) & n > 1 \textit{ for } 1 \leqslant j \leqslant n \end{cases}$$

*The boundary value we can set for B(0) is 1 since we need it to be the same as the base case which is B(1). The reason for that is because we use loop from 1 to n and add up all the values from the multiplication and setting B(0) to 1 allows for a single sided BST. When listing the BSTs that can be generated for n keys we can see that there are cases where there is either no left or right subtree from the root. In cases such as those we still need to compute the value as left subtree number of combinations * right subtree number of combinations. and if there is no left or right subtree and we say that there are 0 combinations then the result for those BST combinations would algorithimically be 0 which doenst make sense. If we were to take the fact that the state of 0 nodes is a BST as well (similar to subsets) then we can say that 0 node can only have 1 unique BST therefore B(0) is 1.*

*Also as stated before the recurrence relies on the combinations of left and right subtrees from 0 & n-1 to 1 & n-2 to n-1 & 0. The number of BSTs for each combination of subtrees can be calculated using the previous computation of number of BSTs for a given n. Since the root does not change for a certain state and only the subtrees change we can calculate the combinations of BST for a state as number of left subtree combinations * number of right subtree combinations.*

(d) (4 pts) Give an iterative (bottom up) algorithm based on the recurrence that computes B(n) by filling in a table.

Algorithm 1 Number of Unique BSTs given Number of Keys

```
1:  function COUNTBST(n)
2:      B ← 1D Array size n + 1
3:      B[0] ← 1
4:      B[1] ← 1
5:      for i = 2 ; i⩽n ; i++ do
6:          for j = 1 ; j⩽i ; j++ do
7:              B[i] ← B[i] + ( B[ i - j ] × B[ j - 1 ] )
8:          end for
9:      end for
10:     return B[n]
11: end function
```

Solution. *This algorithm is a bottom up solution that starts at 0 and 1 nodes and builds up to n nodes*

(e) (1 pt) What is the running time of your iterative algorithm as a function of n (use ? notation)?

Solution.
The running time of the algorithm is $\Theta(n^2)$. *The reason for this is because the main calculation to fill up the DP table is a nested for loop looping from 2 to n and an inner loop that loops to the variable from the first for loop which is another n operations therefore* $\Theta(n) * \Theta(n) = \Theta(n^2)$.

2. (10 pts) Many scientific and engineering applications involve performing long sequences of matrix multiplications, and often the matrices are not square. Matrices M_1 and M_2 can be multiplied if the dimensions of M_1 are $a \times b$ and the dimensions of M_2 are $b \times c$, (so M_1 has the same number of columns as M_2 has rows) and the product will have dimensions $a \times c$. In this problem we will consider only the standard way to multiply two matrices, so multiplying an $a \times b$ matrix with a $b \times c$ matrix takes time abc.

   A sequence of matrix multiplications $M_1 M_2 \cdots M_n$ is defined if each M_i has dimensions $d_{i-1} \times d_i$, so each matrix has a number of columns equal to the next matrix's number of rows. The resulting product will be $d_0 \times d_n$.

   Matrix multiplication is associative, so the sequence of matrices can be parenthesized in any way (but the order of the matrices cannot be changed). Parenthesizing in a clever way can greatly reduce the time required to multiply the matrices. For example consider the following product with $d_0 = 8$, $d_1 = 4$, $d_2 = 10$, and $d_3 = 3$:

$$\underbrace{M_1}_{8\times 4} \cdot \underbrace{M_2}_{4\times 10} \cdot \underbrace{M_3}_{10\times 3}$$

   In this case, parenthesizing as $(M_1 M_2)M_3$ takes time $(8{\cdot}4{\cdot}10){+}(8{\cdot}10{\cdot}3) = 560$ while parenthesizing as $M_1(M_2 M_3)$ takes time $(4 \cdot 10 \cdot 3){+}(8 \cdot 4 \cdot 3) = 216$, and is more than twice as fast. The savings can easily be even more dramatic. You may want to experiment with other sequences of 3 or 4 matrices with varying dimensions. Note that the times to multiply a sequence of n matrices does not depend on the values in the matrices, only on the sequence of dimensions $d_0, d_1, \ldots, d_n$. Furthermore, as each matrix multiplication is done, one of the dimension from the sequence of dimensions is effectively cancelled. The cost of the multiplication is the product of three dimensions: the cancelled dimension and the two adjacent remaining dimensions.

   For this problem you are to create a dynamic programming problem for determining the cost of the best way to parenthesize a sequence of matrix multiplications.

   (a) (4 pts) By considering the last multiplication done, create a recurrence for the minimum cost to multiply a sequence of matrices with dimensions $d_0, \ldots, d_n$ in terms of the minimum costs needed to multiply subsequences of the matrices.

   Solution.

$$C[i][j] = \begin{cases} 0 & i == j \\ min_{i \le k < j}(C[i][k] + C[k+1][j] + d_{i-1} \cdot d_k \cdot d_j) & i < j \end{cases}$$

(b) (1 pt) What is ”smaller” about the subproblems that must be solved?

Solution. *The thing that is smaller about the subproblems is the length of Matricies needed to be multiplied. The matricies M_i and M_j are split into subproblems computing thier products with a matrix M_k between M_i and M_j.*

(c) (4 pts) Give a bottom-up table-filling algorithm for computing the minimum cost to multiply a sequence of matrices from the dimensions $d_0, \ldots, d_n$. Make sure that the values needed to fill in a cell are computed before being used.

Algorithm 2 Optimal Matrix Multiplication Sequence - Cost

```
1:  function MATMULTCOST(d)                                   ▷ d-dimensions array
2:      C ← 2D Array size len(d) x len(d)
3:      for i = 1 ; i < n ; i++ do
4:          C[i][i] ← 0
5:      end for
6:      for l = 2 ; l ⩽ n ; l++ do
7:          for i = 1 ; i ⩽ (n-l+1) ; i++ do
8:              j ← (i+l-1)
9:              min ← INT_MAX
10:             for k = i ; k < j ; k++ do
11:                 temp ← C[i][k] + C[k+1][j] + d[i-1] * d[k] * d[j]
12:                 if temp > min then
13:                     min ← temp
14:                 end if
15:             end for
16:             C[i][j] ← min
17:         end for
18:     end for
19:     return C[1][n]
20: end function
```

Solution.

(d) (1 pt) Briefly describe how the table's values can be used to determine which products should be computed before the last multiplication (i.e. between which M_i and M_{i+1} are separated by the outermost parentheses) in a best way to parenthesize the product. (you need only describe how to determine the "last" multiplication, and need not recursively compute the entire ordering).

Solution. *The last multiplication is stored in C[1][n] so then we need the minimum of* $C[1][k] + C[k+1][n] + d[0] * d[k] * d[n]$. *Since the previous costs were all computed before this one, we can just use the computed values to get the minimum value as the minimum is the most optimal and that is what we need.*

3. (10 points) Read the Coin Changing Problem in the handout on Dynamic Programming and from the lecture notes. Recall this algorithm assumes that an unlimited supply of coins in each denomination $d = (d_1, d_2, \ldots, d_n)$ are available.

   (a) (3 points) Show that this problem exhibits optimal substructure, i.e. show how to divide the problem into subinstances of the same problem in such a way that every subinstance solution is a combination of other subinstance solutions.

   Solution.
   Claim: There exists an optimal substucture S
   Then there is a substructure such that concat(S_1,S_2) = S.

   Assume: S_1 is not optimal. This means that there exists a substructure S_1' that is the most optimal such that concat(S_1',S_2) $<$ concat(S_1,S_2). This contradicts the claim that S is the most optimal substructure.

   Similarly:
   Assume: S_2 is not optimal. This means that there exists a substructure S_2' that is the most optimal such that concat(S_1,S_2') $<$ concat(S_1,S_2). This contradicts the claim that S is the most optimal substructure.

   Then the components of S are optimal solutions to the subproblems due to prro by contradiction.

(b) (2 points) Write a recurrence relation for the value of an optimal solution. Include boundary and out of bounds values.

Solution.

$$C[i][j] = \begin{cases} 0 & j = 0 \\ \infty & i = 1 \qquad j < d_i \\ C[i-1][j] & d[i] > j \\ min\left\{C[i-1][j], 1 + C[i][j - d[i]]\right\} & d[i] \leq j \end{cases}$$

(c) Write a recursive algorithm that, given the filled table $C[1 \ldots n; 0 \ldots N]$ generated by the above algorithm, prints out a sequence of $C[n, N]$ coin types which disburse N monetary units. In the case $C[n, N] = \infty$, print a message to the effect that no such coin disbursal is possible. State why your algorithm is correct. (No need of a formal proof, but make sure that your justification makes sense and is precise. Lengthy answers will be penalised.)

Algorithm 3 Prints Sequence

```
1:  function PRINTSEQUENCE(C, n, N, d)
2:      if C[n][N] == ∞ then
3:          print("IMPOSSIBLE")
4:          return
5:      end if
6:      if N == 0 then
7:          return
8:      end if
9:      if C[n-1,N] == C[n,N] then
10:         printSequence(C, n-1, N, d)
11:     else
12:         print(d[n])
13:         printSequence(C, n, N - d[n], d)
14:     end if
15: end function
```

Solution. *The algorithm is correct because when traversing the table, an infinity value means that the value is not in the table or outside the calculated space. The recursive steps work by checking if the number of coins in the current value in the previous denomination is the same and if so it recurses one denomination less because we want to accuratly represent the minimal number of coins which isnt possible if there is a lesser denomination that can represent the value as well. If the number of coins is different from the current denomination to the previous denomination then it means that the current denomination is the most optimized at the current value then we subtract the denomiation and recurse checking if target value reaches 0 meaning that we have printed all optimal denominations. N=0 only if we print all the optimal denominations that means we subtrated them fromt he target value hitting an even 0.*

4. (10 points) Knapsack.

   (a) (3 points) Show that this problem exhibits optimal substructure, i.e. show how to divide the problem into subinstances of the same problem in such a way that every subinstance solution is a combination of other subinstance solutions.

   Solution.
   Claim: There exists an optimal substucture S
   Then there is a substructure such that concat(S_1,S_2) = S.

   Assume: S_1 is not optimal. This means that there exists a substructure S_1' that is the most optimal such that concat(S_1',S_2) $<$ concat(S_1,S_2). This contradicts the claim that S is the most optimal substructure.

   Similarly:
   Assume: S_2 is not optimal. This means that there exists a substructure S_2' that is the most optimal such that concat(S_1,S_2') $<$ concat(S_1,S_2). This contradicts the claim that S is the most optimal substructure.

   Then the components of S are optimal solutions to the subproblems due to prro by contradiction.

(b) (2 points) Write a recurrence relation for the value of an optimal solution. Include boundary and out of bounds values.

Solution.

$$C[i][j] = \begin{cases} 0 & i = 0 \quad j \geq 0 \\ max(C[i-1][j], v[i] + C[i-1, j-w[i]]) & i > 0 \quad j \geq 0 \\ -\infty & j < 0 \end{cases}$$

(c) (2 points) Write pseudo-code for a dynamic programming algorithm that solves this problem. Your algorithm should take as input the value and weight arrays $v[]$ and $w[]$, and the weight limit W. It should generate a table $V[1 \ldots n; 0 \ldots W]$ of intermediate results. Each entry V[i,j] will be the maximum value of the objects that can be stolen if the weight limit is j, and if we only include objects in the set $1, \ldots, i$. Your algorithm should return the maximum possible value of the goods which can be stolen from the full set of objects, i.e. the value $V[n, W]$. (Alternatively you may write your algorithm to return the whole table.)

Algorithm 4 Maximize Value While Staying in Weight Bounds

```
1:  function KNAPSACK(v,n,w,W)
2:      V ← 2D Array size n x W
3:      for i = 1 ; i < n ; i++ do
4:          V[i][0] ← 0
5:      end for
6:      for i = 1 ; i < W ; i++ do
7:          V[0][i] ← ∞
8:      end for
9:      for i = 1 ; i⩽n ; i++ do
10:         for j = 1 ; j⩽W ; j++ do
11:             if j ≥ w[i] then
12:                 V[i][j] ← max(V[i - 1][j], v[i] + V[i - 1][j - w[i]])
13:             else
14:                 V[i][j] = V[i-1][j]
15:             end if
16:         end for
17:     end for
18:     return V[n,W]
19: end function
```

Solution.

(d) (3 points) Write an algorithm that, given the filled table generated in the last part, prints out a list of exactly which objects are to be stolen.

Algorithm 5 Print Optimal KnapSack

```
1:  function PRINTKNAPSACK(C, n, w, W)
2:      if i == 0 then
3:          return
4:      else
5:          if C[n][W] > C[n-1][W] then
6:              print(n)
7:              printKnapSack(C, n-1, w, W-w[n])
8:          else
9:              printKnapSack(C, n-1, w, W)
10:         end if
11:     end if
12: end function
```

Solution.

5. (10 points) Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is $(5, 10, 3, 12, 5, 50, 6)$. Show the complete tables $m[i, j]$ and $s[i, j]$ described in the handout on Dynamic Programming.

Solution.

$S =$

Weight ↓ Value →	*1*	*2*	*3*	*4*	*5*	*6*
1	*0*	*1*	*2*	*2*	*4*	*2*
2	∞	*0*	*2*	*2*	*2*	*2*
3	∞	∞	*0*	*3*	*4*	*4*
4	∞	∞	∞	*0*	*4*	*4*
5	∞	∞	∞	∞	*0*	*5*
6	∞	∞	∞	∞	∞	*0*

$M =$

Weight ↓ Value →	*1*	*2*	*3*	*4*	*5*	*6*
1	*0*	*150*	*330*	*405*	*1655*	*2010*
2	∞	*0*	*360*	*330*	*2430*	*1950*
3	∞	∞	*0*	*180*	*930*	*1770*
4	∞	∞	∞	*0*	*3000*	*1860*
5	∞	∞	∞	∞	*0*	*1500*
6	∞	∞	∞	∞	∞	*0*

Thus the Optimal parenthesization of Matrix Chain Multiplication is: $(A_1 \cdot (A_2)) \cdot ((A_3 \cdot A_4) \cdot (A_5 \cdot A_6))$